Comparative Analysis of Sorting and Search Algorithms

Jesus Ramirez

El Paso Community College, El Paso, TX 79915, USA

Index Terms— Algorithms, Complexity, Searching, Sorting

I. Introduction

Sorting and search algorithms are fundamental concepts in computer science that allow for the efficient organization and retrieval of data. Their implementation in search engines, databases, and many other systems is crucial for achieving optimal performance and facilitating easier access to information. By understanding the principles of how these algorithms function and perform, we can implement them into our programs to obtain greater performance and pursue the development of more efficient algorithms and software solutions.

This paper aims to analyze and compare eight well-known sorting\search algorithms, focusing on their efficiency in terms of time and space complexity using Big O notation. By evaluating these different algorithms, we can provide insight into their strengths and weaknesses relative to the date we assess them on.

The sorting algorithms tested in this paper include bubble sort, selection sort, insertion sort, merge sort, quick sort, and an algorithm of our choosing, explained in Section III. The two search algorithms that will be tested include linear search and binary search. Their evaluation will be based on how long each algorithm takes to perform its specified function so we can provide performance results and a comparative analysis.

The remainder of this paper is structured as follows: Section II outlines our objective and testing approach. Section III identifies our sixth algorithm selection. Section IV explains each algorithm's theoretical performance. Section V details our exact test cases. Section VI presents the experimental results of our tests. Section VII summarizes the contents of our experiment. Section VIII includes the code used to conduct our tests.

II. Objective and our approach

Our general objective is to analyze how each algorithm performs its given task. More specifically, for the sorting algorithms, we will strictly focus on the time it takes to sort a list in only ascending order. The search algorithms will be evaluated based on the time spent finding the first occurrence of a specific element in a list. Each algorithm will also be implemented in Java and executed in VS Code.

To effectively analyze the efficiency of each algorithm, we will be testing with different variations of data to capture their performance given different test cases. Our data types will consist of a list containing strings and another list consisting of integers. Using these two data types, we can examine the difference in the time it takes to compare strings in contrast to comparing integers and how it affects the total time of an algorithm. Both lists will also be assorted in different arrangements, e.g., unsorted, sorted in reverse order, and uniform data, to identify the strengths and weaknesses of an algorithm.

By incorporating a broad range of data, we can thoroughly analyze how and why an algorithm operates in a different test case and identify its advantages and disadvantages.

III. Sixth algorithm selection

The sixth sorting algorithm chosen for this paper is radix sort, specifically LSD (least significant digit) radix sort, which implements a non-comparative approach to sorting, unlike the other five algorithms.

A. Description of Radix Sort

Instead of comparing elements, LSD radix sort works by focusing on the digits or characters of each element. Starting with the least significant digit/character (rightmost digit/character) to the most significant digit/character (leftmost digit/character), each pass groups elements into buckets based on the current digit/character being examined. After processing all digits/characters, the elements are collected from the buckets in order, resulting in a sorted list [1].

B. History of Radix Sort

The first implementation of a radix sort dates back to the late 19th century with Herman Hoffling and his development of tabulating machines. These machines used a form of radix sorting to effectively sort data from punched cards up to the 20th century. At the time, computerized radix sorts were deemed impractical because of their unreliable memory allocation, which could vary in size and was therefore unpredictable until 1954, when Harold H. Seward at MIT developed the first memory-efficient computerized radix sort algorithm. Today, radix sort algorithms have been refined and adapted to effectively sort collections of strings, integers, and other data types [2].

C. Complexity

Radix sort has an average time complexity of $\Theta(n * k)$ where n is the number of elements in a list and k is the number of digits/characters of the largest element. The best case scenario

for radix sort is also $\Omega(n * k)$ as well as for the worst case scenario of $O(n * k)$. The space complexity of radix sort is on average $O(n + k)$.

IV. Performance of Each Approach

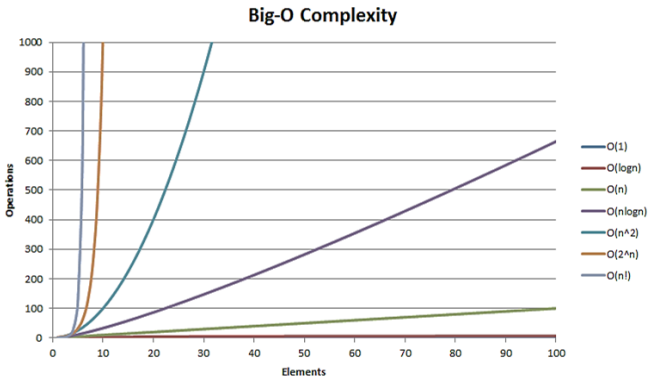


Figure 1: demonstrates different common time complexities and how the growth rate differentiates from each function [7].

A. Bubble Sort

Bubble sort is a simple sorting algorithm that repeatedly traverses a list and sorts adjacent items if they are in the wrong order until sorted. As bubble sort iterates through the list, the largest element in the iteration is brought to the end. Bubble sort is most useful in small data sets as it becomes impractical with larger sets of data.

Bubble sort usually has two nested for-loops where the outer loop iterates $n - 1$ times, causing the inner loop to run $(n - 1)$ times where *n* is the number of elements in a list. Multiplying the work of both loops results in $\theta\big((n - 1)(n - 1)\big)$ which simplifies to the average time complexity of $\theta(n^2)$. Therefore, as *n* increases, the time and operations needed to sort a list increase quadratically.

Using a modified bubble sort to detect if no swaps were made while running, the best case scenario for bubble sort is $\Omega(n)$, given a list already sorted in ascending order. This is because the outer loop only iterates one time, and the inner loop iterates $n - 1$ times to compare elements then terminates. The work of both loops can be expressed as $\Omega(1 * n - 1)$, simplifying to a linear time complexity of $\Omega(n)$.

The worst case scenario for bubble sort occurs when a list is given in descending order resulting in the outer loop iterating $n - 1$ and the inner loop to run $(n - 1) + (n - 2) + \cdots + 1$ times. The number of operations bubble sort performs in the worst case is the summation of the inner loop which can be expressed as $\sum_{i=1}^{n-1}(n - i) = n(n - 1)/2 = (n^2 - n)/2$, which simplifies to a time complexity of $O(n^2)$.

Aside from the memory allocated for variables, bubble sort does not require extra space as it sorts a list in place, resulting in a constant space complexity of $O(1)$.

Algorithm 1 Pseudocode for Optimized Bubble Sort

```
start function bubbleSort (A : list of sortable items)
    set n to length of A
    repeat
        set swapped to false
        for j = 0 to n – 1 do
            if A[ j ] is greater than A[j + 1] then
                swap A[ j ] with A[j + 1]
                set swapped to true
            end if
        end for
    repeat until not swapped
end function
```

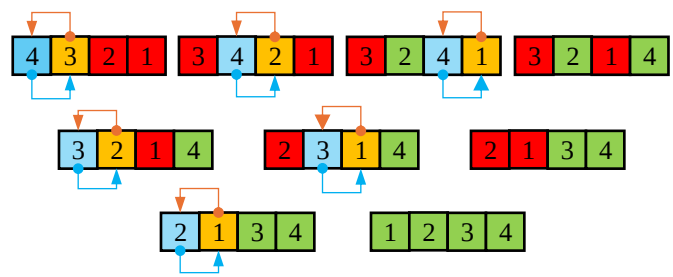


Figure 2: demonstrates bubble sort, with each level depicting an iteration of the outer loop. In the visualization, blue denotes the current index of the inner loop, orange represents the element being compared to the blue element, red indicates elements in the incorrect position, and green signifies elements in the correct position. Arrows indicate swaps between elements according to their color.

B. Selection Sort

Selection sort is another simple sorting algorithm that repeatedly finds the smallest element in the unsorted part of the list and swaps it with the element at the beginning of the unsorted sublist, which is determined by the index *'i'* of the outer loop. Mainly used for instructional purposes and sorting lists of small sizes, selection sort becomes impractical with larger data sets.

Selection sort is an inefficient algorithm with a quadratic time complexity of $O(n^2)$ for the best, average, and worst cases. For every case, the outer loop will always iterate $n - 1$ times, and the inner loop will always iterate $(n - 1) + (n - 2) + \cdots + 1$ times which simplifies to $n(n - 1)/2 = O(n^2)$.

What makes selection sort even more ineffective is that it will always perform the same number of operations for every case relative to the size of a list and iterations of the loops. Therefore, every time the inner loop iterates $(n - 1) + (n - 1) + \cdots + 1$ times, it will always make $n(n - 1)/2$ number of comparisons.

Similarly every iteration of $n-1$ of the outer loop will always result in $n-1$ number of swaps.

Like bubble sort, aside from the memory allocated for the variables, selection sort sorts the list in place, which equates to a constant space complexity $O(1)$.

Algorithm 2 Pseudocode for Selection Sort

```
start function selectionSort (A : list of sortable items)
    set n to length of A
    for i = 0 to n - 1 do
        set smallestItem to i
        for j = i + 1 to n do
            if A[ j ] is less than A[ smallestItem ] then
                set smallestItem to j
            end if
        end for
        swap A[ i ] with A[ smallestItem ]
    end for
end function
```

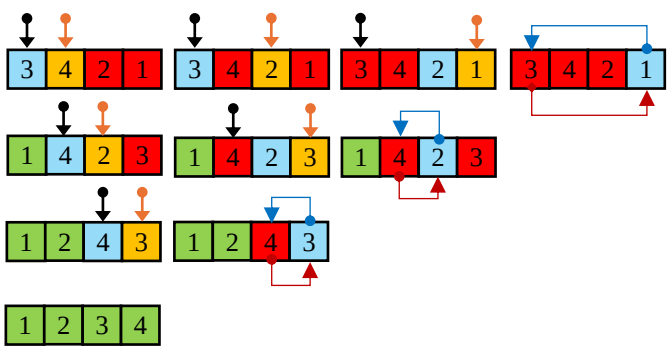


Figure 3: demonstrates selection sort, with each level representing an iteration of the outer loop. The black arrow indicates the current index of the outer loop, The orange arrow indicates the current index of the inner loop, blue denotes the current smallest element found, orange represents the element being compared to blue, red indicates elements in the incorrect position, and green signifies elements in the correct position. Arrows indicate swaps between elements according to their color.

C. *Insertion Sort*

Insertion sort is the last of these introductory algorithms that are more suitable for small data sets than large ones. It sorts elements by iteratively traversing a list, comparing and shifting each element with those to its left until finding the correct position for insertion.

Similar to the previous two algorithms, Insertion sort has an average time complexity of $\Theta(n^2)$, where n is the number of elements in a list, as it also utilizes two loops where the outer loop iterates $n-1$ times, and the inner loop will at most run $(n-1)$ times. Essentaily we can multiply both loops as $\theta\big((n-1)(n-1)\big)$ simplyfing to $\theta(n^2)$.

Given a list in ascending order, the outer for loop would iterate $n-1$ times, and the while loop will perform $n-1$ comparisons. Since the number of comparisons and iterations of the outer loop correspond to each other, we can express these complexities as $\Omega\big((n-1)+(n-1)\big)$ which would simplify to a linear time complexity of $\Omega(n)$.

Like the past other algorithms, the worst case for insertion occurs when it is given a list sorted in descending order. Resembling bubble sort, the outer loop will iterate $n-1$ times resulting in the inner loop to run $1+\cdots+(n-2)+(n-1)$. The number of comparisons and swaps can also be calculated by the summation of the times the while loop runs as $\sum_{i=0}^{n-1} n-1 = n(n-1)/2 = (n^2-n)/2$ which simplifies to a time complexity of $O(n^2)$.

This is the last of the algorithms that sorts a list in place and only needs to memory to allocate for variables which means it has a constant space complexit of $O(1)$.

Algorithm 3 Pseudocode for Insertion Sort

```
start function insertionSort (A : list of sortable items)
    set n to length of A
    for i = 1 to n do
        set j to i
        while j > 0 and A[ j ] < A[j - 1] do
            swap A[ j ] with A[j - 1]
            decrement j by 1
        end while
    end for
end function
```

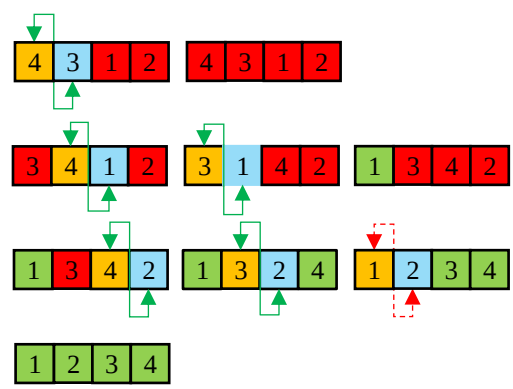


Figure 4: demonstrates inseriton sort, with each level representing an iteration of the outer loop. Blue denotes the current element being sorted, orange represents the current element being compared to blue, green arrows signifys a true comparison and swap, red arrow signifys a false comparison with no swap, red indicates elements in the incorrect position, and green signifies elements in the correct position. Arrows indicate swaps between elements according to their color.

D. *Merge Sort*

Now getting into the more advanced algorithms, Merge sort is a recursive divide and conquer sorting algorithm as it breaks down a list into small manageable subsets. It works by recursevly continually splitting a list and the resulting sublists until every element is its own sublist. It then merges the sublists in order until the list is fully sorted.

Merge sort has a time complexity of $O(n \log n)$ for the best, average, and worst cases making it a suitable program for large sets of data. It repedeatly divides the list by $n/2, n/4, n/8 \ldots, 1$ where n is the number of elements. The total number of divisions or levels in a run is $\log_2 n$ so if n = 8, the number of levels would be $\log_2 8$ or 3. For every split, the number of sublist in a level is 2^i where i is the current level of recursion, and the number of elements in each sublist is around $n/2^i$. So if n = 8 and we are in the second level of recursion, the number of sublist would be 2^2 which is 4, and the number of elements is $8/2^2$ or 2 elements in each list. The total number of sublist can be expressed as the summation of 2^i as i reaches $\log_2 n$ such as $\sum_{i=1}^{\log_2 n} 2^i$ so again using n = 8, the total number of sublist would be $2^1 + 2^2 + 2^3 = 14$ total sublist. So since in for each level of i there are 2^i sublist and every sublist has aroud $n/2^i$ elements, the number of operations can be calcualted by $(n/2^i) * 2^i$ which would equal n operations for each level. Therefore, for every level of $O(\log_2 n)$ there are around $O(n)$ operations which results in our time complexity of $O(n \log_2 n)$ [6].

The space complexity of merge sort is $O(n)$ as every recursive call will will create temporary lists to store the sorted version and copy it to the original.

Algorithm 3 Pseudocode for Merge Sort

```
start functtion mergeSort(A : list of sortable items)
    if length of A is not 1 then
        set middle to mid index of A
        pass mergeSort(A_start - middle)
        pass mergeSort(A_middle - end)
        merge(A from A_left and A_right)
    end if
end function

start function merge(A, A_left, A_right)
    T : temporary array to store sorted list
    while A_left and A_right have elements do
        if A_left[currIndex] < A_right[currIndex] then
            set T[currIndex++] to A_left[currindex++]
        else
            set T[currIndex++] to A_lrigth[currindex++]
        end if
    end while
    add remaining elements from A_left to T
    add remaining elements from A_right to T
    copy elements from T to corresponding A pos
end function
```

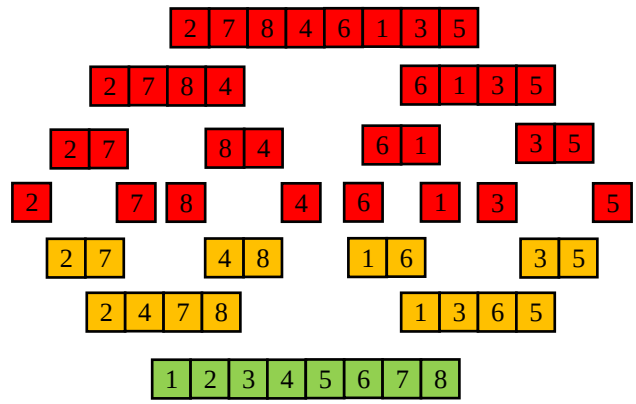


Figure 5: demonstrates merge sort, with each level representing a level of recursion. Red indicates the dividing stage, orange signifys the merging stage, and green represents the completed array.

E. *Quick Sort*

Quick Sort is another recursive divide and conquer algorithm that is also effectice on high data sets. Quick sort works by selecting an element as a pivot which will be a reference to put elements less than the pivot to the left, and those greater to the right. There are different ways to select the pivot but for this paper, the last element of the list will serve as our pivot.

Quick sort on average has a time complexity of $O(n\log n)$ resembling merege sort where it divides a list and its sublist until they are in there own list. On average, quick sort will have $O(\log n)$ levels of recusrsion with each level requiring arorud n operations which is giving us $O(n\log n)$. The main difference between quick sort and merge sort, is that merge sort will always divide every list in half wheras a list in quicksort will be based on the pivots final position.

Since our split reference will be the pivots final point, quick sort has a time complexity of $O(n^2)$ when the list is sorted in either ascending or descending. This is because the pivot will always result in being the largest or smallest number causing every division of a list where one sublist will contain the pivot and the other will contain all the other elements

On avaerage has a space complexity of $O(\log n)$ and in worst case of $O(n)$.

Algorithm 4 Pseudocode for Quick Sort

```
A : list of sortable items
start function quicksort(A : list of sortable items)
    if length of A not 0 then
        set pivot to returned partition(A) value
        quicksort(A_(start - (pivot - 1)))
        quicksort(A_((pivot + 1) - end))
    end if
end function

start function partition(A, left, right)
    newpivot = left - 1
    pivot = a[right] : last element of A
    for leftpos = left until leftpos = right do
        if a[leftpos] < pivot then
            swap(a[++newpiv], a[leftpos])
        end if
    end for
    swap(a[++newpiv], a[leftpos])
    return newpivot
end function
```

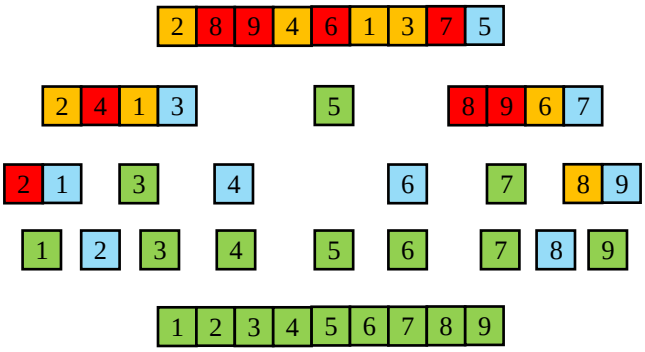


Figure 6: demonstrates quick sort, with each level representing a level of recursion. Blue denotes the current pivot in the level, orange represents elements smaller than our pivot, red represents elements larger than our pivot, and green signifies elements in the correct position.

F. Radix Sort

As mentioned in Section III, radix sort is a non-comparative sorting algorithm that works by inspecting each digit or character of every element, and sorting them based on the digit/character being inspected.

The best, and average, and worst case scenario for radix sort is $O(n * k)$ where n is the number of elements and k is the largest element. Our *n* comes from the number of elements we are going to iterate and *k* determines the number of iterations done for each element.

The space complexity if $\mathrm{O}(\mathrm{n}+\mathrm{k})$ as for every iteration, radix sort creates a temporary lists, one to store the sorted array and another to keep track on the number of occurrences of an elements digit/character.

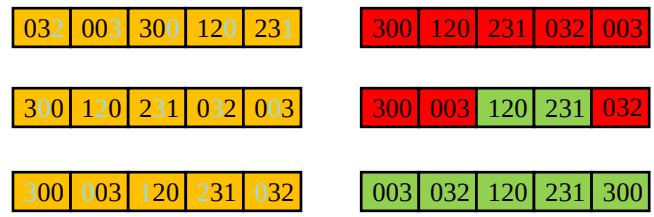


Figure 7: demonstrates radix sort, with each level representing a level of iteration. Orange represents the comparison stage, numbers in blue indicate the current element being inspected, **red indicates elements in the incorrect position, and green signifies elements in the correct position.**

G. Linear Search

Now getting into the search algorithms of this paper, linear search is a search algorithm that works by checking every element in a list until it finds that element or the end of the list is reached. Linear search has an advantage over the other search algorithms in this paper, given a list that is unsorted.

The best case for linear search occurs when the element we are trying to find is at the beginning of the list resulting in the for loop running 1 time and doing one comparison resulting in a constant time complexity of $\Omega(1)$.

In the average and worst case the time complexity is O(n) as the algorithm will either find the element somewhere in the list besides the first index, or if the element is not in the list.

Linear search requires no extra space and has a space complexity of O(1).

Algorithm 6 Pseudocode for Linear Search

```
num : number to be found
start function linearSearch(A : list of sortable items)
    for each element in A do
        if element = num then
            return num in list A
        end if
    end for
    return num not in list A
end function
```

H. Binary Search

Binary search is a more effective sorting algorithm given that the list is already sorted in ascending order. Binary search works by choosing the middle element in the list and seeing if it's the element we are trying to find, or if it's greater or less than the middle element.

Best case of O(1) if the element we are tring to find is in the middle of a list.

Average and worst case share the same time complexity of $O(\log n)$ as the list will continiusly divide until it reaches only one element.

Algorithm 7 Pseudocode for Binary Search

```
num : number to be found
start function binarySearch(A : passed list, num)
    if length of A is 0 then
        return num not in A
    end if
    if n = middle element of A then
        return n found in A
    else if  n < middle element of A then
        binarySearch(A_{start – middle}, num)
    else num > middle element of A then
        binarySearch(A_{middle – end}, num)
    end if
end function
```

V. TESTING STRATEGY

A. What Data to Sort

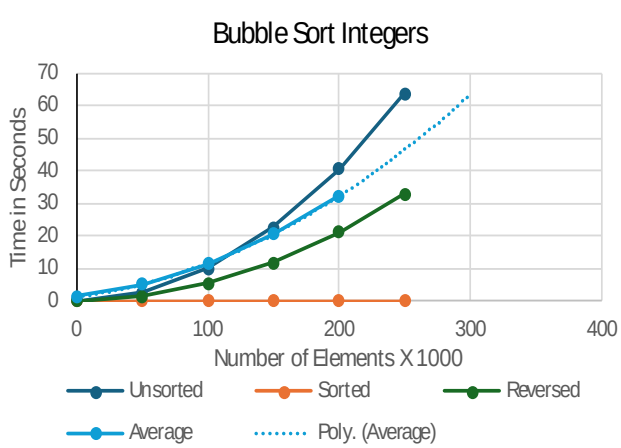
For this lab, we are given a text file named "dictionary.txt" containing 641,392 words consisting of English and Spanish words with some special characters included. We will also be testing with another text file named "numbers.txt" which will consist of numbers ranging from 1–100 million.

B. Variations

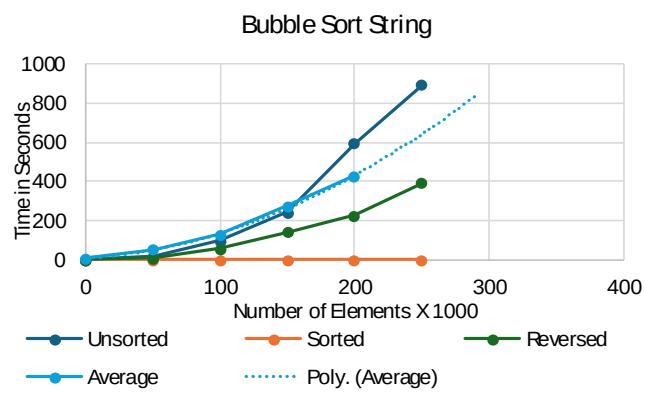
For both files of data, we will be testing different variations of arrangements. Variations will include both files already being sorted, being sorted in reverse order, and randomly being shuffled every run.

VI. EXPERIMENTS AND RESULTS

A. Bubble Sort

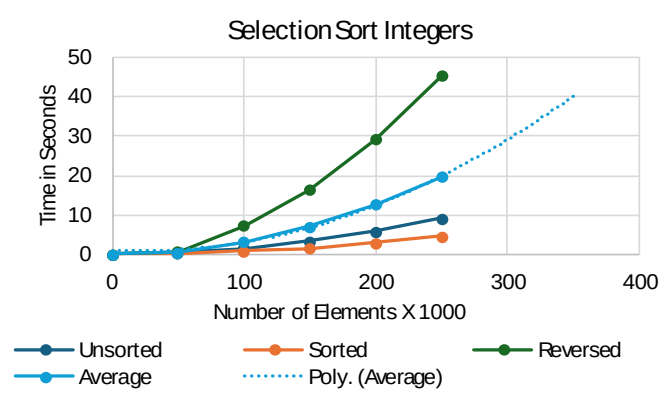


Looking at the result from bubble sort in sorting integers, it performed quite actually as expected. The three test cases used were passing an unsorted list, an already sorted list in ascending order, and another in descending order. There were five trials for each variation starting from 50,000 elements to 250,000 elements. In the unsorted list trials, bubble sort appears to form a quadratic function which is exactly its time complexity. Using an optimized bubble sort to detect if no swaps were made, it also performed as expected when given a sorted list. For the worst case of an unsorted list, it performed suprisengly well as the modifed bubble sort will sort the list halfway through causing it to break early.

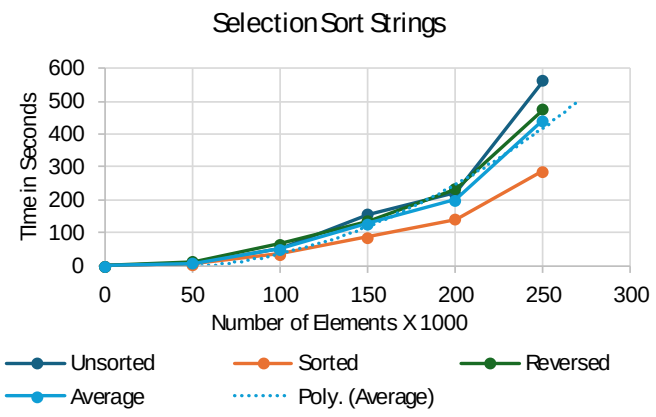


Looking at the result from bubble sort in sorting Strings, it follows the same pattern as sorting integers but it did take more time as comparing strings takes longer. The three test cases used were passing an unsorted list, an already sorted list in ascending order, and another in descending order. There were five trials for each variation starting from 50,000 elements to 250,000 elements. In the unsorted list trials, bubble sort appears to form a quadratic function which is exactly its time complexity. Using an optimized bubble sort to detect if no swaps were made, it also performed as expected when given a sorted list. For the worst case of an unsorted list, it performed suprisengly well as the modifed bubble sort will sort the list halfway through causing it to break early.

B. Selection Sort

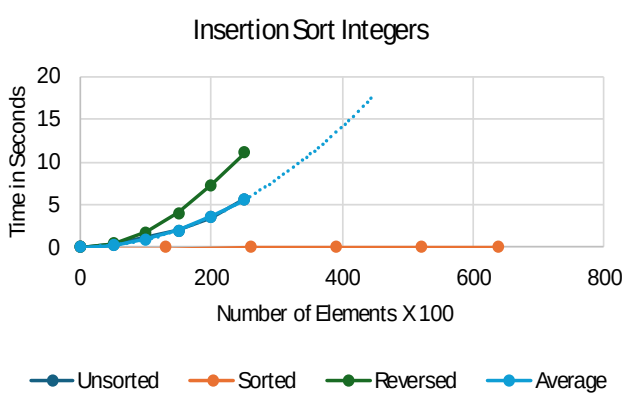


Selection Sort was the worst algorithm to work with as it took the longest time to sort both strings and integers. There were five trials for each variation starting from 50,000 elements to 250,000 elements. This one also performed as expected where the worst case would be a descending list causing it to perform all operations. Selection sort performed better with a sorted list, but it is still to close to the runtimes of the unsorted list trials.

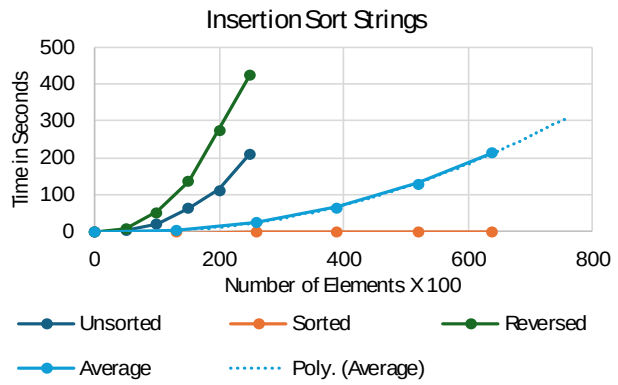


Selection Sort appears to have worked better with strings but in reality, We were not able to perform any further test cases after 250,000 elements as it was way to slow. The three test cases used were passing an unsorted list, an already sorted list in ascending order, and another in descending order. There were five trials for each variation starting from 50,000 elements to 250,000 elements.

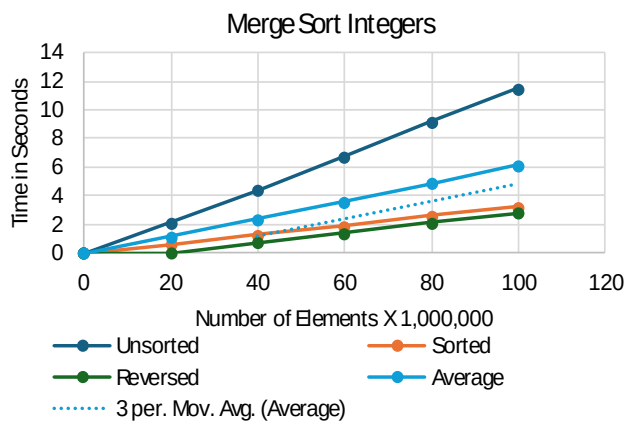
C. *Insertion Sort*



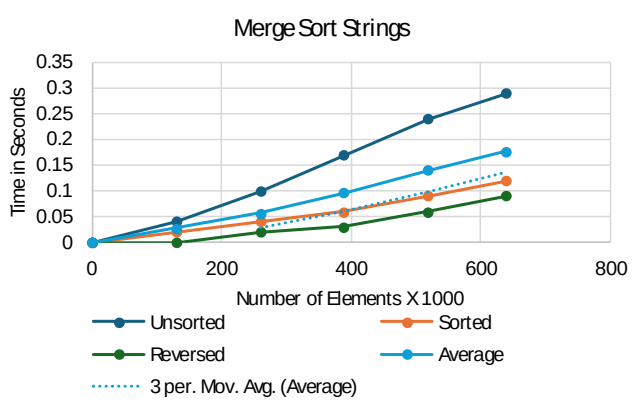
For insertion sort, there were also three test cases used inclding passing an unsorted list, an already sorted list in ascending order, and another in descending order. There were five trials for each variation starting from 50,000 elements to 250,000 elements. Insertion Sort, like bubble sort, performed well in a sorted list as it only had to check if they were in the right place. It did however perform worse with the reversed list, and it performed as expected with the unsorted list as it approaches a quadratic function.



D. *Merge Sort*

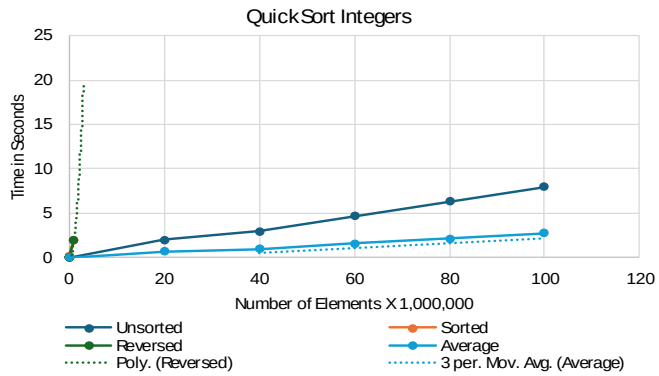


For merge sort and sorting integers, we conducted on a list of integers containing numbers from 1 – 100 million. As shown in the graph below, merge sort did extremely well with its highest time being 11.52 seconds on an unsorted list.

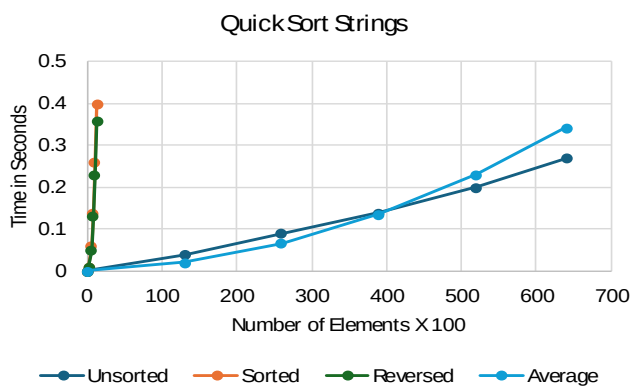


For sorting a list of strings it did even better as we would have expected it to take longer than integers as comparing strings takes more time. For merge sort and sorting string, we conducted on a list of strings going in intervals of 130,000 until reaching 641,392 strings.

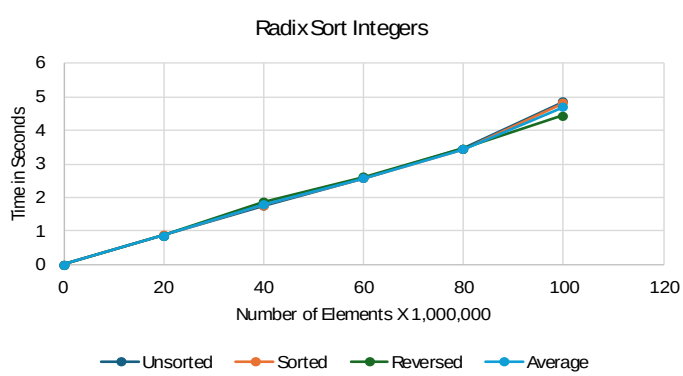
E. *Quick Sort*



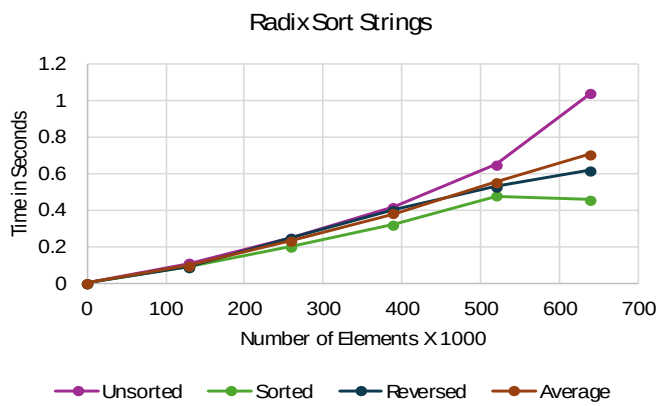
For quick sort and sorting integers and strings, we also used numbers from 1 – 100 millions, and a varying range of strings from 2,600 to 641,392. In the case of unsorted lists, quick sort performed better than merge sort. However, since we selected our pivot as the last element, it resulted in in the worst case complexity of $O(n^2)$ as it would always pick the highest or lowest element every single recursive level causing the sublists to be split unevenly. For both strings and integers, we could not test with more than 13,000 elements as it would result in a stack overflow.



F. *Radix Sort*



Now getting into out algorithm selections, radix sort had the greatest efficeiny when it came to sorting intgers as its max time for a list of one hundred million elements was only 4.86 seconds. Radix sort was also efficeint in sorting strings from 130,000 to 641,392 string with its max time being 1.04 seconds. Not as well as merge sort but it was a very unique algorithm to implements.



G. *Search Algorithms*

For the search algorithms, there was essentially no significant difference in seconds between linear and binary search. For linear search, we tested finding a random number in a list from 1 – 100 million, and finding the five words in the dictionary file. At most that most time to find either elements was 0.02 seconds. For binary search, looking for a random number and a string from the dictionary file all resulted in 0 seconds with a max time of 0.01 seconds for either data type.

VII. Conclusion

Summarizing the results of our experiment, it is evident that the first three algorithms are more usefull to teach the foundations of sorting algorithms as they become highly impractical with large data sets. For merge sort and quick sort, they have a better implementation in large data sets and also teaches the foundations of recursion. For the quadratic sorting algorithms, sorting strings takes a lot longer as the string comparisons takes more time. And for the other two algorithms, They have are much more efficient in sorting that the time of string comparisons becomes negligable compared to the time complexity of the entire algorithm.

As for our search algorithms, even traversing a lost of one hundred million elements is not enough to notice a time difference in our experiments. Although for data one hundred times larger than that will of course benefit from using a logarithmic search algorithm such as binary search. The only scenario where linear search has the upper hand is when it would take much longer to sort a set of data rather than traversing the entire thing.

VIII. Code Implementations

Code implementations are in the qr code below as the code got way to big to post in the document.

IX. References

[1] GfG, “Radix sort - data structures and algorithms tutorials,” GeeksforGeeks,https://www.geeksforgeeks.org/radix-sort/ (accessed Mar. 27, 2024).

[2]“Radixsort,”Wikipedia,https://en.wikipedia.org/wiki/Radix_sort (accessed Mar. 27, 2024).

[3] Adrian Mejia, “How to find time complexity of an algorithm?,” Adrian Mejia Blog, https://adrianmejia.com/how-to-find-time-complexity-of-an-algorithm-code-big-o-notation/ (accessed Mar. 27, 2024).

[4] SimSim 59111 gold badge55 silver badges1414 bronze badges, jmadjmad 9, JoeJoe 4, kushjkushj 12133 bronze badges, and user84630user846301, “Evaluating the average time complexity of a given bubblesort algorithm.,” Computer ScienceStackExchange,https://cs.stackexchange.com/questions/20/evaluating-the-average-time-complexity-of-a-given-bubblesort-algorithm (accessed Mar. 27, 2024).

[5] Comparison among selection sort, bubble sort, and insertion sort,https://www.interviewkickstart.com/learn/comparison-among-bubble-sort-selection-sort-and-insertion-sort (accessed Mar. 27, 2024).

[6] V. Khandelwal, “What is merge sort algorithm: How does itwork,andmore,”Simplilearn.com,https://www.simplilearn.com/tutorials/datastructuretutorial/mergesortalgorithm#:~:text=with%20practical%20examples.,What%20Is%20a%20Merge%20Sort%20Algorithm%3F,sublists%20into%20a%20sorted%20list. (accessed Mar. 27, 2024).

[7] “Big O cheatsheet - data structures and algorithms with thier complexitiesvarunNR,”HackerEarth,https://www.hackerearth.com/practice/notes/big-o-cheatsheet-series-data-structures-and-algorithms-with-thier-complexities-1/ (accessed Mar. 27, 2024).